# AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 29: Over-Fitting

Adam Prugel-Bennett

November 2, 2026

## 1 Keywords

- Overfitting, regularisation, feature selection, early stopping, margins, double descent

## 2 Main Points

- Over-fitting
  - Over-fitting is when we learn **spurious rules** that explain the training set
  - If we use an infinitely flexible machine then we can't generalise at all
- Controlling Complexity
  - More training examples improve generalisation by allowing us to eliminate spurious rules
  - To achieve good generalisation performance machine must match the underlying structure of the problem
  - By preprocessing or careful feature engineering you can sometimes make the learning task easier
  - Deep learning works partly because it uses a lot of training data and partly because architectures such as convolutional networks match the structure of the data (see the next lecture)
  - We usually don't know the structure of the inputs (they are too high dimensional)
    - Try different machines to see what fits
    - Fine-tune the models (adjusting hyper-parameters)
    - Feature engineering very often helps
      - Feature selection
      - Projecting onto a few principal components (PCA)
      - Normalise your input features
    - Need a validation set to do this
    - Beware you are very likely to over-estimate your generalisation error if you use your test set to select your model
    - Can use $K$-fold cross-validation to get a better estimate of generalisation error
- Regularisation
  - Adding regularisation terms can help choose a simpler model
  - $L_2$ regularisers punish large weights making the fitting function smoother; $L_1$ regularisers (the lasso) can do automatic feature selection

- **Regularisers are priors.** The MAP solution maximises $\log f(\mathcal{D}|\boldsymbol{w}) + \log f(\boldsymbol{w})$, so $-\log f(\boldsymbol{w})$ acts as a regulariser. Gaussian noise of variance $\sigma^2$ with a Gaussian prior $w_i \sim \mathcal{N}(0, s^2)$ gives ridge regression with $\eta = \sigma^2/s^2$; a Laplace prior $f(w_i) \propto \mathrm{e}^{-|w_i|/b}$ gives the lasso penalty $\eta \sum_i |w_i|$
- **Early stopping is implicit ridge.** Gradient descent on $\frac{1}{2}\|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2$ with learning rate $r$ is $\boldsymbol{w} \leftarrow \boldsymbol{w} - r\,\mathbf{X}^\mathsf{T}(\mathbf{X}\boldsymbol{w} - \boldsymbol{y})$. With the SVD $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$, component $k$ of $\boldsymbol{w}$ along $\boldsymbol{v}_k$ obeys $w_k \leftarrow (1 - r s_k^2)\, w_k + r\, s_k\, \boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}$. Starting from $\boldsymbol{w} = \mathbf{0}$, after $t$ steps

$$w_k(t) = \left(1 - (1 - r\, s_k^2)^t\right) \frac{\boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}}{s_k}$$

  where $\boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}/s_k$ is the least squares component (see the exercises). Ridge regression multiplies the same component by $s_k^2/(s_k^2 + \eta)$. Both factors are near 1 for large $s_k$ and near 0 for small $s_k$, where the variance lives, so stopping after $t$ steps acts roughly like ridge with $\eta \approx 1/(r\,t)$
- **Why SVMs generalise.** For data in a ball of radius $R$ separated with margin $\gamma$, the capacity of the separator depends on $(R/\gamma)^2$ rather than on the dimension of the feature space (Vapnik), which is why the kernel trick, with enormous or infinite feature spaces, need not over-fit. Also, removing a training point that is not a support vector leaves the solution unchanged, so the leave-one-out error is at most (number of support vectors)$/m$

## 2.1 Double Descent

- Least squares on $m = 40$ points using the first $p$ of 100 features: the test error shows the classical U-curve, a spike at $p = m$, and then a second descent (Figure 1)

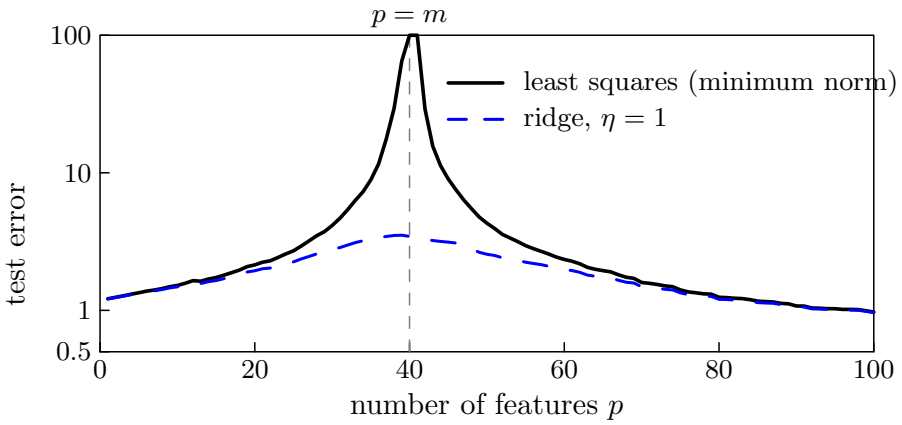



Figure 1: Test error of least squares (minimum norm for $p > m$) and of ridge regression against the number of features

- At $p = m$ there is exactly one fit through every noisy point. The smallest singular value of $\mathbf{X}$ is then close to 0, and the variance $\sum_k \sigma^2/s_k^2$ (lecture on bias and variance) explodes
- For $p > m$ there are infinitely many exact fits. We take the one with the smallest norm, $\hat{\boldsymbol{w}} = \mathbf{X}^\mathsf{T}(\mathbf{X}\mathbf{X}^\mathsf{T})^{-1}\boldsymbol{y}$
- Gradient descent from $\boldsymbol{w} = \mathbf{0}$ finds exactly this solution (see the exercises): the optimiser itself regularises
- Modern neural networks have far more parameters than training examples and still often generalise well; counting parameters is not the same as measuring complexity (Belkin et al., 2019)

# 3 Exercises

## 3.1 Estimating the error in the mean

- Consider a binary classification problem
- Suppose we measure the accuracy on a validation set of size $n = 10\,000$
- If we correctly predict $8\,000$ of the validation set what is
  1. an estimate of the mean accuracy?
  2. the accuracy of this estimate?
- State you assumptions
- What would the answer be if $n = 100$ and we got 80 correct on the validation set?
- Answer at the end

## 3.2 Early stopping

- Derive the formula for $w_k(t)$ above from the update $w_k \leftarrow (1 - r s_k^2)\, w_k + r\, s_k\, \boldsymbol{u}_k^\mathsf{T} \boldsymbol{y}$ with $w_k(0) = 0$. What happens as $t \to \infty$ if $0 < r\, s_k^2 < 2$?

## 3.3 Minimum norm

- Show that every gradient descent step on $\frac{1}{2}\|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2$ adds a vector of the form $\mathbf{X}^\mathsf{T}\boldsymbol{a}$. Hence, starting from $\boldsymbol{w} = \mathbf{0}$, explain why the limit for $p > m$ is the minimum-norm exact fit.

# 4 Experiments

## 4.1 Overfitting Game

- Write your own simulation (see lecture notes)
- Generate $n$ random numbers $X_i \sim U(0, 3)$ (you can try different distributions)
- Add some noise to generate $Y_i = X_i + Z_i$ where $Z_i \sim \mathcal{N}(0, 1)$
- Choose $i^*$ with the largest value of $Y_i$
- Compute $\Delta = Z_{i^*} = Y_{i^*} - X_{i^*}$
- Repeat many times and plot a histogram

## 4.2 Double descent

- Fit least squares (minimum norm when $p > m$) with $m = 40$ points and $p$ of 100 features, and print the median test error over many training sets. (The mean is useless at $p = m$: the expected error is infinite there.)

```
import numpy as np

rng = np.random.default_rng(0)
D, m, sigma, trials = 100, 40, 0.5, 200
beta = rng.standard_normal(D) / np.sqrt(D)
for p in [10, 30, 38, 40, 42, 60, 100]:
    err = []
```

```
    for _ in range(trials):
        X = rng.standard_normal((m, D))
        y = X @ beta + sigma * rng.standard_normal(m)
        w = np.linalg.lstsq(X[:, :p], y, rcond=None)[0]   # min-norm if p > m
        err.append(((w - beta[:p])**2).sum() + (beta[p:]**2).sum() + sigma**2)
    print(p, round(np.median(err), 2))
```

### 4.3 Cross-validation

- Try using cross validation

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn import datasets
from sklearn import svm

X, y = datasets.load_iris(return_X_y=True)
X.shape, y.shape

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=0)

X_train.shape, y_train.shape

X_test.shape, y_test.shape


clf = svm.SVC(kernel='linear', C=1).fit(X_train, y_train)
clf.score(X_test, y_test)
```

### 4.4 Practical work

- There are a lot of practical tips that you can use in your project
    - Try them out and see what works and what doesn't
    - Many modifications won't make much difference, but occasionally one will make a lot of difference
    - Be very wary of estimating your generalisation performance
        - You are measuring on a finite validation set so you will have errors
        - Once you've used your validation set for selecting models, you are likely to over-estimate your generalisation performance (under-estimate the generalisation error)

## 5 Answers

### 5.1 Estimating the error in the mean

- We assume that the examples in the validation set are independent so the number of successes, $k$, will be binomially distributed

$$\mathrm{Bin}(k|n,p) = \binom{n}{k} p^k \left(1-p\right)^{n-k}$$

- We don't know the success probability $p$, but an unbiased estimate of it is the number of observed success divided by $n$, that is $0.8$

- For a binomial the variance in $k$ is given by $n\,p\,(1-p)$ or 1600 if $p = 0.8$ and $n = 10\,000$
- The typical size of the fluctuations in $k$ would be $\sqrt{n\,p\,(1-p)} = 40$
- Our estimate of $p$ would therefore typically have fluctuations of size $40/n = 0.004$
- Thus our estimated success rate is $0.8 \pm 0.004$ or 80% with an error of about half a percent
- This is small because we used a large validation set. If $n = 100$ and we got 80 successes then we would have an error of $0.8 \pm 0.04$ so we have an error of 4%.
- Note that by choosing the best of 10 machines we might expect to over-estimate the performance by around 5% in this case

### 5.2 Early stopping

- Let $c_k = s_k\,\boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}$ and $a = 1 - r s_k^2$. Then $w_k(t) = c_k\,r\,(1 + a + \cdots + a^{t-1}) = c_k\,r\,(1-a^t)/(1-a) = (1-a^t)\,c_k/s_k^2 = (1-a^t)\,\boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}/s_k$
- If $0 < r s_k^2 < 2$ then $|a| < 1$, so $a^t \to 0$ and $w_k$ converges to the least squares component $\boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}/s_k$. Small $s_k$ means $a$ close to 1 and slow convergence: these are the last directions to be fitted

### 5.3 Minimum norm

- The step is $-r\,\mathbf{X}^\mathsf{T}(\mathbf{X}\boldsymbol{w} - \boldsymbol{y})$, of the form $\mathbf{X}^\mathsf{T}\boldsymbol{a}$. Starting from $\mathbf{0}$, $\boldsymbol{w}$ always stays in the space spanned by the rows of $\mathbf{X}$
- For $p > m$ (and $\mathbf{X}\mathbf{X}^\mathsf{T}$ invertible) the limit fits the data exactly, so $\boldsymbol{w} = \mathbf{X}^\mathsf{T}\boldsymbol{a}$ with $\mathbf{X}\mathbf{X}^\mathsf{T}\boldsymbol{a} = \boldsymbol{y}$, giving $\boldsymbol{w} = \mathbf{X}^\mathsf{T}(\mathbf{X}\mathbf{X}^\mathsf{T})^{-1}\boldsymbol{y}$
- Any other exact fit is $\boldsymbol{w} + \boldsymbol{z}$ with $\mathbf{X}\boldsymbol{z} = \mathbf{0}$. Then $\boldsymbol{z}$ is orthogonal to every row of $\mathbf{X}$, hence to $\boldsymbol{w}$, so $\|\boldsymbol{w} + \boldsymbol{z}\|^2 = \|\boldsymbol{w}\|^2 + \|\boldsymbol{z}\|^2 \geq \|\boldsymbol{w}\|^2$: gradient descent finds the minimum-norm fit

### 5.4 Double descent

- The median test errors are about 1.47 ($p = 10$), 3.89 ($p = 30$), 14.8 ($p = 38$), 86 ($p = 40$), 16.4 ($p = 42$), 2.2 ($p = 60$) and 0.98 ($p = 100$): a peak at $p = m$ and a second descent that ends below anything the small models achieve